

UNIVERSIDAD CATÓLICA DEL URUGUAY

Facultad de Ingeniería y Tecnologías

Ingeniería en Informática

Informe de Proyecto

Algoritmos y Estructuras de Datos

Equipo 3

Santiago Acosta

Milagros García

Federico Martínez

Bruno Pereda

Alexis Giménez

9 de septiembre de 2026

Índice

1. Introducción	1
2. Descripción general de la solución	1
2.1. Biblioteca de estructuras jerárquicas	1
2.2. Modelado de la sala de emergencias	2
2.3. Operaciones principales del sistema	2
3. Relación con el primer hito	3
4. Estructuras utilizadas e invariantes	4
5. Principales decisiones de diseño	5
6. Análisis de complejidad	6
7. Decisiones tomadas en el Desafío 2	8
8. Optimización realizada en el Desafío 3	10
9. Resultados de los experimentos realizados	11
10. Declaración de uso de herramientas de IA	12
11. Conclusión	13

1. Introducción

El presente informe corresponde al segundo hito del Proyecto Integrador de Algoritmos y Estructuras de Datos. En el primer hito se construyó una biblioteca propia de estructuras lineales y se usó para modelar la gestión de una sala de emergencias. En este segundo hito se hizo una extensión de esta biblioteca con estructuras jerárquicas (árbol binario, árbol binario de búsqueda, árbol AVL, árbol general y montículo binario) que son incorporadas al sistema para poder cumplir de manera correcta con los requisitos que las propias estructuras lineales no eran capaces de resolver de manera adecuada.

El principal cambio que hay en el escenario es el paso de un registro de hechos sueltos a un registro que mantiene la relación de origen en cada atención. Ahora al ingresarse un paciente, se abre un episodio en el cual cada estudio, interconsulta, procedimiento o complicación es precedido por aquello que lo motivó, sin conocer de antemano cuántos niveles de derivación pueden haber. A esto se le suma un catálogo institucional de diagnósticos organizado por capítulos, grupos y códigos, la posibilidad de modificar la prioridad de un paciente que ya está esperando y la consulta de atenciones por rango de fechas sin recorrer todos los registros.

A lo largo del informe se describe la solución construida, se explica qué se mantuvo del primer hito y qué fue reemplazado, se presentan las estructuras utilizadas junto con los invariantes que cada una mantiene y se analiza el costo de las operaciones principales en comparación con el costo que tenía la misma operación en la entrega anterior. También se detallan las decisiones que fueron tomadas en el Desafío 2, la optimización medida y sus resultados en el Desafío 3. Por último, se detalla el uso que de las herramientas de IA y se dan las conclusiones del equipo.

2. Descripción general de la solución

2.1. Biblioteca de estructuras jerárquicas

El Desafío 1 consistió en extender la biblioteca del primer hito con las estructuras jerárquicas dadas por las interfaces del propio proyecto. Se implementaron árbol binario, ABB AVL árbol genérico y Heap. Las tres primeras forman una cadena de herencia: ABB extiende árbol binario agregándole el orden entre claves y, AVL extiende ABB agregándole lo relacionado al balanceo.

Sobre los árboles binarios se implementaron los cuatro recorridos: preorden, inorden y postorden los cuales se resuelven de forma recursiva sobre el propio nodo, a diferencia de el recorrido por niveles que se apoya en la lista enlazada del primer hito y que es usada como cola de pendientes: se agrega la raíz y, en cada paso, se retira el primer elemento, se lo visita y se encolan sus hijos. Como lista enlazada mantiene una referencia a su último nodo, tanto agregar al final como retirar del frente tienen un costo de $O(1)$, y el recorrido completo queda en $O(n)$. ABB ofrece además una consulta por rango, que baja el árbol y descarta las ramas que quedan fuera de las cotas pedidas. Esta operación es la que sostiene las consultas por fecha de la sala.

Árbol genérico representa el árbol n -ario de la siguiente manera: cada elemento guarda su dato, una referencia a su primer hijo y otra a su hermano siguiente, con lo cual un nodo con n cantidad de hijos se sostiene con dos referencias fijas. Además de los recorridos, ofrece operaciones sobre subárboles, que son las que después usan Episodio y CatalogoDiagnosticos. Por último heap implementa el montículo binario sobre una array list, con el criterio de orden dado como Comparator. También tiene insertar, eliminar (el cual retira el mínimo), mínimo y remover que sirve para quitar un elemento cualquiera.

2.2. Modelado de la sala de emergencias

El modelo parte de las mismas entidades que ya tenía el primer hito y agrega las que pide el nuevo escenario. Paciente conserva los atributos que tenía y suma dos elementos: una referencia al episodio en curso, que es null cuando no tiene ninguno abierto y un AVL con todos sus episodios tanto abiertos como cerrados, ordenados por fecha de apertura. Los cinco niveles de urgencia y su tiempo máximo de espera tolerable quedan sin cambios. En los estados, el único cambio fue pasar de EN_CONSULTORIO a EN_ATENCION ya que el consultorio dejó de existir.

La atención se representa mediante Episodio y EventoClinico. Un Episodio tiene un identificador, paciente al que pertenece, fecha de apertura y cierre y un estado. Contiene también un árbol genérico de eventos cuya raíz es la consulta inicial.

El EventoClinico guarda identificador, identificador del paciente, su tipo, su descripción, fecha en que ocurrió, una lista enlazada de insumos, una lista enlazada con los códigos de diagnóstico asociados, un estado y su fecha de cierre. Los tipos de evento posibles son cinco: CONSULTA_INICIAL, ESTUDIO, INTERCONSULTA, PROCEDIMIENTO y COMPLICACION.

Tanto el episodio como cada evento usan el mismo enum de estado, con los valores ABIERTO y CERRADO. Un Insumo tiene nombre, costo unitario y cantidad consumida, calcula también su costo total como el producto de estos últimos. Los insumos se registran en el evento donde se consumen de manera única por lo que el costo de un episodio nunca toma el mismo insumo dos veces.

El catálogo institucional de diagnósticos contiene un árbol genérico de sus propios nodos (Nodo-Catalogo). Cada nodo tiene un código, nombre y un nivel dado por el enum NivelCatalogo, que tiene como valores: RAIZ, CAPITULO, GRUPO y CODIGO.

RAIZ existe para un único nodo artificial que sostiene a los capítulos y que se oculta por fuera, los otros tres corresponden a una clasificación real. Solo los nodos de nivel código pueden asociarse a un evento como diagnóstico, y el evento guarda únicamente dicho código como String ya que, la clasificación vive en el catálogo.

SalaEmergencia sigue siendo el componente coordinador del flujo de información. Mantiene el registro general de pacientes en un AVL ordenado por id, la espera en un Heap ordenado por el comparador POR_URGENCIA, un AVL de episodios y otro de eventos clínicos ordenados por fecha y, el catálogo de diagnósticos. Ahora cada paciente lleva también su propio índice de episodios de manera que la misma información histórica puede consultarse tanto a nivel de sala como a nivel de una persona puntual.

2.3. Operaciones principales del sistema

Registrar un paciente, buscarlo por identificador, eliminarlo y listarlos mantienen la misma interfaz del primer hito solo que ahora se resuelven sobre AVL. Agregar un paciente a la cola le asigna una urgencia pasa a EN_ESPERA y lo inserta en el heap. Se agregó cambiarPrioridad que saca al paciente del heap cambia su urgencia y lo devuelve y, proximoAA atender que devuelve el paciente de la raíz.

El ingreso a la atención se hace con abrirEpisodio o con atenderSiguiente. La primera recibe el identificador de un paciente y crea el episodio con su consulta inicial como raíz. La segunda toma al más urgente del heap y luego abre su episodio. En ambos casos el episodio se registra en el paciente y en el AVL de episodios de la sala, la consulta inicial queda en el AVL de eventos y el paciente pasa a estar en atención. Un paciente no puede tener dos episodios abiertos al mismo tiempo pero sí puede volver a abrir uno cuando el anterior se cerró.

registrarEvento cuelga un evento nuevo de aquel que lo originó, agregarInsumo carga un insumo

en un evento concreto y diagnosticar asocia a un evento un código del catálogo, verificando que exista y que sea de nivel código. `cerrarEvento` cierra un evento si todo lo que se desprendió de él está cerrado. Cuando no puede, `queImpideCerrarEvento` devuelve la lista de descendientes que siguen abiertos. `cerrarEpisodio` aplica esa misma regla sobre la consulta inicial y si sigue adelante, deja al paciente en atendido. Los costos y los tiempos se consultan con `costoDe` y `duracionAcumuladaMinutos` para cualquier evento, y con `costoDelEpisodioEnCurso` para el episodio completo.

Por último, el sistema ofrece las consultas que se apoyan en el orden de los árboles: `episodiosEnRango` y `eventosEnRango` sobre toda la sala, la sobrecarga de `episodiosEnRango` con identificador de paciente para el historial de una persona y, sobre el catálogo, `buscarEnCatalogo` y `codigosDelCatalogo`, que devuelve todos los códigos diagnosticables que preceden a un capítulo, un grupo o un código. El método `toString` de `SalaEmergencia` resume la cantidad de pacientes registrados, en espera, episodios, eventos y códigos del catálogo.

3. Relación con el primer hito

Al estar especificado que la solución extienda lo construido sin romper su comportamiento, la primera tarea fue definir qué partes del primer hito mantener. Se conservaron las estructuras lineales de la biblioteca junto con sus pruebas al varias de ellas seguir en uso dentro de la sala. Las características de cada paciente, los insumos y los códigos de cada evento se guardan en una lista enlazada. Lista enlazada también se usa como cola de pendientes en el recorrido por niveles y como contenedor de resultados de las consultas. `ListaArray` es el soporte interno del Heap y es usada en el benchmark para armar las listas de ids. También se conservaron el comparador `POR_URGENCIA`, que el paciente se identifique a través de `equals` y `hashCode` de su identificado, los niveles de urgencia, la validación mediante excepciones y el criterio de que la información histórica sobreviva a la eliminación del paciente.

Primero se decidió reemplazar el registro general de pacientes. En el primer hito era una `ListaArray` ordenada por identificador, sobre la que `buscarPaciente` aplicaba búsqueda binaria en $O(\log n)$. Lo que quedó pendiente fue que el registrar y eliminar seguían costando $O(n)$ lo cual al cambiarse por un AVL, las tres operaciones descienden un único camino del árbol y quedan en $O(\log n)$.

La espera de atención pasó de una `ColaPrioridad` a un Heap. `ColaPrioridad` era una lista enlazada que se mantenía ordenada al insertar, con frente en $O(1)$ y cada inserción en $O(n)$. El heap cambia esa relación: insertar y extraer el mínimo cuestan $O(\log n)$ y consultar el frente sigue en $O(1)$. El motivo de peso fue el requerimiento nuevo de modificar la prioridad de un paciente que ya espera, que se resolvió con una operación `remove` en el TDA seguida de una reinserción. A cambio se perdió el desempate por orden de llegada entre pacientes de igual urgencia, que la lista ordenada daba de forma natural y que el heap no garantiza.

El historial de consultas, que era una Pila, y los consultorios, que eran una `ListaArray` de capacidad ampliable, dejaron de existir como estructuras de la sala. La Pila permitía ver la última consulta en $O(1)$, pero no ofrecía ningún orden aprovechable para preguntar qué pasó entre dos fechas: había que recorrerla entera. En su lugar, el historial se reparte en dos AVL de la sala, uno de episodios y otro de eventos, ambos ordenados por fecha, y en el AVL de episodios de cada paciente. Los consultorios se eliminaron porque el episodio ya representa la etapa de atención en curso, y el estado `EN_CONSULTORIO` se renombró a `EN_ATENCION` para reflejar ese cambio. La clase `Consulta` fue sustituida por `Episodio` y `EventoClinico`, que además de registrar qué se hizo registran qué lo originó.

Las clases `ColaPrioridad` y `Pila` siguen formando parte de la biblioteca, junto con sus pruebas, aunque la sala ya no las utilice. De esta forma no se quita ninguna funcionalidad ya entregada, las pruebas del primer hito siguen ejecutándose sobre el mismo código que validaron entonces, y

queda disponible, dentro del mismo proyecto, la implementación lineal que sirve de referencia para la comparación del Desafío 3.

4. Estructuras utilizadas e invariantes

AVLImpl es la estructura de los tres índices principales de la sala (pacientes, episodios y eventos) y del índice de episodios de cada paciente. Mantiene dos invariantes. El primero es el de todo árbol binario de búsqueda: para cada nodo, todas las claves del subárbol izquierdo son menores y todas las del derecho son mayores, sin duplicados, ya que insertar rechaza un dato que ya se encuentra según su criterio de orden. El segundo es el de balance: en cada nodo, la diferencia entre la altura del subárbol izquierdo y la del derecho es a lo sumo uno en valor absoluto. Cada AVLElemento guarda su altura, que se recalcula al volver de cada inserción y eliminación, y cuando el balance de un nodo llega a dos se aplica una rotación simple o doble según el caso. Ese invariante es el que acota la altura del árbol en $O(\log n)$ y el que sostiene el costo de todas las operaciones que descienden por un camino.

El orden de cada AVL lo define el `compareTo` de su tipo. Paciente compara por identificador; Episodio y EventoClinico comparan por fecha y, si dos coinciden, por identificador, de modo que dos episodios abiertos en el mismo instante conviven en el árbol sin que uno anule al otro. Las búsquedas usan Comparable parciales (Paciente.`porId`, EventoClinico.`porId`, Episodio.`porFecha` y EventoClinico.`porFecha`) que saben compararse con un elemento sin necesidad de construir uno completo.

Heap sostiene la espera de atención. Su invariante es el de orden del montículo: para todo nodo, el elemento del padre es menor o igual que el de cada uno de sus hijos según `POR_URGENCIA`, es decir, según el tiempo máximo de espera tolerable de su urgencia. Como consecuencia, el paciente más urgente está siempre en la raíz. La estructura es un árbol binario completo guardado por niveles en una `ListaArray`, donde los hijos del elemento en la posición i están en $2i+1$ y $2i+2$ y su padre en $(i-1)/2$, así que no hay referencias entre nodos y el árbol nunca degenera. Insertar agrega al final y hace flotar el elemento mientras sea menor que su padre; eliminar mueve el último a la raíz y lo hunde, y en cada paso elige al menor de sus hijos; `remove` hace lo mismo desde la posición del elemento quitado, y lo flota o lo hunde según corresponda.

ArbolGenerico tiene dos usos con invariantes distintos, y en ambos casos las reglas del dominio las hace cumplir la clase que contiene al árbol y no el árbol en sí, que solo garantiza que cada nodo cuelgue del padre indicado y que no haya datos repetidos. En Episodio el invariante es que todo evento cuelga de aquel que lo originó, que la raíz es siempre la consulta inicial y que un evento cerrado no admite hijos nuevos. Episodio.`registrarEvento` verifica que el episodio esté abierto, que el evento pertenezca al mismo paciente, que el padre exista y que siga abierto antes de insertar. De ahí se desprende una propiedad que el cierre aprovecha: si un evento está cerrado, todo su subárbol está cerrado, porque `cerrarEvento` se niega a cerrar mientras haya descendientes abiertos y ningún hijo puede agregarse después.

En CatalogoDiagnosticos el invariante es el de la clasificación: un capítulo cuelga únicamente de la raíz, un grupo únicamente de un capítulo y un código únicamente de un grupo, y ningún código se repite en el catálogo. Lo hace cumplir el método privado `agregarBajo`, que busca al padre y comprueba que su `NivelCatalogo` sea el esperado antes de delegar en el árbol, mientras que el rechazo de duplicados lo aporta ArbolGenerico.`insertar`. Con este invariante, las hojas del subárbol de un capítulo o de un grupo son exactamente sus códigos diagnosticables, que es lo que devuelve `codigosBajo`.

Además de los invariantes de cada estructura, la sala mantiene algunos que atraviesan varias de ellas. Un paciente tiene a lo sumo un episodio abierto, y esa referencia, `episodioActual`, es la que usan todas las operaciones sobre el episodio en curso. Todo evento que forma parte de un

episodio está también en el AVL global de eventos, porque registrarEvento inserta en ese índice solo cuando el árbol del episodio aceptó el evento. Y un paciente cuyo estado es EN_ESPERA está en el heap, lo que permite que abrirEpisodio decida por el estado si tiene que retirarlo de la espera.

5. Principales decisiones de diseño

Una de las principales decisiones fue representar el episodio de atención mediante un árbol general y no mediante una estructura lineal. En el primer hito cada atención se guardaba como una Consulta suelta en el historial, lo que permitía saber qué se había hecho pero no qué había originado qué. El escenario planteado describe otra cosa: la consulta inicial deriva en estudios, un estudio puede motivar una interconsulta y un procedimiento puede tener complicaciones. Como ni la cantidad de derivaciones ni su profundidad se conocen al abrir el episodio, ninguna representación de aridad fija resulta adecuada. Internamente el árbol se implementa con la representación de primer hijo y hermano derecho, de modo que un nodo con una cantidad arbitraria de hijos se sostiene con dos referencias fijas, igual que un nodo binario. Llegar al hijo número k obliga a recorrer los hermanos anteriores, pero el sistema nunca necesita un hijo en una posición determinada.

También se decidió que Episodio contuviera un árbol y no que fuera un árbol. Episodio no extiende ArbolGenerico ni lo expone hacia afuera; ofrece operaciones propias del dominio, como registrar un evento, cerrarlo o consultar su costo, y resuelve cada una apoyándose en los recorridos del árbol. El catálogo de diagnósticos sigue el mismo criterio.

Los insumos se registran en el evento donde efectivamente se consumen y no en el episodio. Cada EventoClinico mantiene su propia lista de insumos, y el costo de cualquier parte del episodio se obtiene recorriendo en postorden el subárbol correspondiente y sumando lo que se consumió en cada nodo. De esta manera la información no queda repetida en los niveles intermedios, y una misma operación informa el costo de una complicación puntual o el del episodio completo. El recorrido es en postorden porque el valor de un nodo depende del de sus descendientes. Los tiempos se tratan igual, con la diferencia de que solo se acumulan los eventos ya cerrados.

La regla de cierre se resolvió como una propiedad del subárbol. Un evento puede cerrarse únicamente cuando todo lo que se desprendió de él está cerrado, por lo que queImpideCerrar recorre su subárbol y devuelve la lista de descendientes que siguen abiertos. Se prefirió devolver esa lista antes que un valor de verdad, porque el sistema debe indicar qué impide cerrar un episodio y un booleano no lo explicaría. Cerrar el episodio completo se expresa entonces como cerrar su raíz, ya que el subárbol de la consulta inicial es el episodio entero. A esto se agregó una restricción complementaria: un evento cerrado no admite derivaciones nuevas, porque representa una intervención terminada.

Para el catálogo institucional de diagnósticos se utilizó nuevamente un árbol general, ya que la clasificación por capítulos, grupos y códigos es jerárquica por definición. La decisión particular fue agregar una raíz que no forma parte del catálogo y que existe para sostener los capítulos como hijos, porque sin ella el árbol tendría tantas raíces como capítulos. Esa raíz se oculta hacia afuera y cantidadNodos la descuenta. El enumerado NivelCatalogo permite validar que un grupo se cuelgue de un capítulo y un código de un grupo, y que solo los nodos de nivel CODIGO se usen como diagnóstico. Consultar por cualquier nivel se resuelve pidiendo las hojas del subárbol correspondiente, así que consultar un capítulo completo no obliga a enumerar a mano los códigos que contiene. El evento guarda solamente el código, ya que el catálogo es el único lugar donde vive la clasificación.

Respecto a las búsquedas, se decidió expresar los criterios como Comparable parciales en lugar de exigir el objeto completo. Paciente por Id, EventoClinico por Id y Episodio por Fecha devuelven

un comparable que sabe ubicarse frente a un elemento sin ser uno de ellos. Con esto se puede buscar un paciente conociendo solamente su documento, y una estructura puede consultarse con un criterio que mira solo una parte del criterio que define su orden, que es lo que permite pedir por fecha un árbol de episodios ordenado por fecha y, en caso de empate, por identificador.

El registro general de pacientes pasó de la ListaArray ordenada del primer hito a un árbol AVL. La búsqueda ya costaba $O(\log n)$ gracias a la búsqueda binaria, pero mantener el orden obligaba a desplazar elementos en cada inserción y dejaba el registro en $O(n)$. Con el AVL la búsqueda se mantiene en $O(\log n)$ y la inserción y la eliminación pasan a ser logarítmicas, porque conservar el orden exige a lo sumo una cantidad acotada de rotaciones. El balanceo fue el motivo principal de la elección: un árbol binario de búsqueda sin balancear degenera en una lista cuando los identificadores se registran en orden creciente, que es el caso más probable. Los episodios y los eventos se indexan en árboles AVL ordenados por fecha de apertura, y la consulta por rango descarta las ramas que quedan fuera del intervalo, así que su costo depende de la altura del árbol y de la cantidad de elementos informados. Los episodios se indexan además dentro de cada Paciente.

La espera pasó de la ColaPrioridad implementada sobre una lista ordenada a un montículo binario. La cola anterior consultaba al primer paciente en $O(1)$, pero pagaba $O(n)$ en cada incorporación, porque debía recorrer la estructura hasta la posición que le correspondía al nivel de urgencia. Con el heap, insertar y extraer cuestan $O(\log n)$ y consultar el próximo paciente sigue costando $O(1)$. Pesó también el requerimiento de modificar la prioridad de un paciente que ya está esperando, que se resuelve removiéndolo del heap y volviéndolo a insertar; por este motivo se incorporó al TDA una operación para remover un elemento cualquiera. A cambio se aceptó perder el desempate por orden de llegada entre pacientes de igual urgencia, ya que el heap no es una estructura estable.

El heap usa `Comparator<Paciente>` y no `Comparable`. Fue una decisión explícita: Paciente ya tiene su orden natural por identificador (para el AVL de registro), y la urgencia es un criterio de orden completamente distinto, situacional, que no debería vivir dentro de Paciente. En vez de forzarlo a implementar dos criterios de orden o cambiar su `Comparable`, el heap recibe `POR_URGENCIA` como `Comparator` inyectado desde `SalaEmergencia`. Separa quién es este paciente (identidad) de qué tan urgente es ahora (prioridad), que son cosas independientes: un paciente no cambia de identidad cuando cambia de urgencia.

Una optimización puntual en `abrirEpisodio`: solo se llama a `esperaAtencion.remove(paciente)` si el paciente efectivamente está `EN_ESPERA`. Sin ese chequeo, cada apertura de episodio pagaba un recorrido $O(n)$ del heap aunque el paciente nunca hubiera pasado por la cola, por ejemplo cuando el episodio se abre directamente para un paciente recién registrado o para uno que ya fue atendido y vuelve. Conviene tenerlo presente en el análisis de complejidad de esa operación, porque sin el chequeo `abrirEpisodio` sería $O(n)$ en todos los casos en vez de $O(\log n)$ en el caso típico.

Finalmente, el episodio absorbió dos elementos que en el primer hito estaban separados. Los consultorios, representados con una ListaArray, y el historial de consultas, guardado en una Pila, dejaron de existir como estructuras propias, y `EN_CONSULTORIO` se reemplazó por `EN_ATENCION`. Se mantuvo, en cambio, el criterio del primer hito sobre la información histórica: los episodios y los eventos se registran en índices propios de la sala, así que un paciente puede salir del registro activo sin que se pierdan sus atenciones.

6. Análisis de complejidad

Como en el primer hito, conviene distinguir el costo de una operación dentro de una estructura del costo total de una operación de la sala, ya que una misma operación del sistema puede tocar

varias estructuras y el costo dominante no siempre está en la estructura nueva. En lo que sigue, n es la cantidad de pacientes registrados, m la cantidad de eventos de un episodio en particular, V la cantidad total de eventos de la sala, E la cantidad total de episodios, C la cantidad de nodos del catálogo y k la cantidad de resultados que devuelve una consulta. Para cada operación se indica además en qué se diferencia del costo que tenía la operación equivalente en el primer hito.

`registrarPaciente` cuesta $O(\log n)$. `AVLImpl.insertar` hace primero una búsqueda para rechazar duplicados y después desciende por un único camino hasta la hoja donde corresponde el nuevo nodo; al volver, recalcula alturas y aplica a lo sumo una rotación simple o doble, cada una en $O(1)$. Como el árbol se reequilibra en cada inserción, la altura queda siempre acotada en $O(\log n)$. `buscarPaciente` también cuesta $O(\log n)$, porque la búsqueda desciende por un único camino acotado por la altura. En el primer hito la búsqueda ya era $O(\log n)$ por búsqueda binaria, así que el AVL no la mejora; lo que mejora es el registro, que pasa de $O(n)$, por el desplazamiento de los elementos posteriores, a $O(\log n)$.

`eliminarPaciente` es $O(n)$ en total, aunque el AVL resuelva su parte en $O(\log n)$. La operación busca al paciente, lo retira del heap con `esperaAtencion.remove` y recién después lo elimina del AVL. `Heap.remove` ubica el elemento con una búsqueda lineal, porque el montículo está organizado por prioridad y no por identidad, y esa búsqueda domina el costo. En el primer hito la eliminación también era $O(n)$, pero por otro motivo: el recorrido lineal de `indiceDe` sobre el arreglo y el desplazamiento posterior. Cuando la espera está vacía, como ocurre en el benchmark del Desafío 3, el costo efectivo se reduce al del AVL.

Respecto a la cola de espera, `agregarPacienteACola` cuesta $O(\log n)$: se localiza al paciente en el AVL y se lo inserta en el heap, donde el nuevo elemento se agrega al final y flota comparándose con su padre, con un máximo de tantos pasos como altura tiene el montículo, que es $O(\log n)$ por ser un árbol completo. `proximoAA atender` es $O(1)$, porque el más urgente está siempre en la raíz. Extraer al siguiente con `Heap.eliminar` es $O(\log n)$: se reemplaza la raíz por el último elemento y se lo hunde como máximo la altura del heap. En el primer hito la ColaPrioridad tenía el frente en $O(1)$ y la inserción en $O(n)$. El heap invierte esa relación y, sobre n inserciones, el costo acumulado pasa de $O(n^2)$ a $O(n \log n)$, a cambio de que extraer el mínimo pase de $O(1)$ a $O(\log n)$.

`cambiarPrioridad` es $O(n)$, no $O(\log n)$. `Heap.remove` tiene que encontrar primero al paciente, y como el heap no tiene orden por identidad esa búsqueda es lineal; recién después reacomoda el hueco en $O(\log n)$ y la reinserción cuesta otro $O(\log n)$. La búsqueda domina el costo total. Es una operación nueva que el primer hito no ofrecía, y su costo es el precio de no mantener un índice adicional que relacione cada paciente con su posición en el arreglo.

`abrirEpisodio` cuesta $O(\log n + \log E + \log V)$ en el caso típico: localiza al paciente en el AVL de pacientes, inserta el episodio en el AVL del paciente y en el de la sala, e inserta la consulta inicial en el AVL de eventos. Si el paciente está `EN_ESPERA` se agrega el $O(n)$ de retirarlo del heap con `remove`, que pasa a dominar. `atenderSiguiente` extrae al más urgente en $O(\log n)$ y luego llama a `abrirEpisodio`; como el estado del paciente sigue siendo `EN_ESPERA` hasta que `abrirEpisodio` lo actualiza, ese camino paga también el recorrido de `remove`, y el total queda en $O(n)$. En el primer hito la operación equivalente, `ingresarPaciente`, era $O(n)$ por la eliminación en la lista enlazada de la cola.

`registrarEvento` cuesta $O(m)$ más $O(\log V)$. `Episodio.registrarEvento` busca el evento padre en el árbol del episodio, y esa búsqueda es $O(m)$ porque el árbol general no tiene orden entre hermanos y no admite poda. `ArbolGenerico.insertar` hace además una búsqueda previa para rechazar duplicados, también $O(m)$, y luego recorre los hermanos del padre para colgar el nuevo nodo al final. Una vez aceptado, el evento se inserta en `eventosPorFecha` en $O(\log V)$. `cerrarEvento`, `queImpideCerrar`, `costoDe` y `duracionAcumuladaMinutos` cuestan $O(m)$: son

recorridos en postorden desde el evento pedido, y hay que visitar cada descendiente para verificar su estado o sumar su costo, porque eso es exactamente lo que exige la consigna. `cerrarEpisodio` combina la búsqueda del paciente con el cierre de la raíz, $O(\log n + m)$. En el primer hito no existía la jerarquía; `darDeAlta` apilaba una Consulta en $O(1)$, pero no verificaba nada porque no había nada que verificar. Una lista tampoco habría servido: recorrerla para sumar costos costaría lo mismo, $O(m)$, pero sin conservar quién derivó de quién.

Las búsquedas en el catálogo son $O(C)$ sobre el total de nodos, y conviene decirlo con claridad: no son logarítmicas. El árbol general no tiene un criterio de orden entre hermanos, los capítulos y los grupos no están ordenados entre sí, así que ubicar un código obliga a recorrer el árbol hasta encontrarlo. `buscarNodo` y `diagnosticar` cuestan $O(C)$, y `diagnosticar` suma la búsqueda del evento en el episodio, $O(m)$. `codigosBajo` busca el nodo en $O(C)$ y después recorre su subárbol para juntar las hojas, con un costo proporcional al tamaño de ese subárbol. Lo que el catálogo sí resuelve, que es lo que pidió la dirección médica, es no tener que enumerar los códigos a mano: `codigosBajo` con un capítulo trae todos los códigos que contiene sin que nadie los liste uno por uno. En el primer hito el catálogo no existía; la única forma de responder esa pregunta habría sido mantener una lista plana de todos los códigos y filtrarla entera cada vez.

Las consultas por rango de fechas son la comparación más fuerte con el primer hito. `episodio-sEnRango` y `eventosEnRango` cuestan $O(\log E + k)$ y $O(\log V + k)$ respectivamente: `enRango` desciende el AVL y, si un nodo queda fuera de la cota inferior, ni siquiera visita su subárbol izquierdo; si queda fuera de la superior, no visita el derecho. La sobrecarga `episodiosEnRango` con identificador de paciente encadena dos podas, la búsqueda del paciente en $O(\log n)$ y el rango sobre su propio AVL en $O(\log e + k)$, con e la cantidad de episodios de ese paciente. En el primer hito el historial era una Pila sin ningún orden aprovechable, y consultar por rango exigía recorrer el historial completo revisando uno por uno si cada elemento caía en el intervalo, $O(n)$ sobre la cantidad de consultas. Aislar las atenciones de un paciente puntual tampoco tenía forma de resolverse sin ese recorrido.

Las operaciones de listado siguen siendo lineales, igual que antes: `listarPacientes` y `listarEpisodios` recorren su AVL en inorden, $O(n)$ y $O(E)$, con la diferencia de que ahora la salida sale ordenada por identificador o por fecha sin ningún trabajo adicional. El recorrido por niveles de un árbol visita cada nodo una vez, $O(n)$, ya que agregar al final y retirar del frente de la `ListaEnlazada` cuestan $O(1)$.

En cuanto al espacio, todas las estructuras son lineales en la cantidad de elementos. Cada nodo de `AVLImpl` guarda el dato, dos referencias y un entero con la altura; cada nodo de `ArbolGenerico` guarda el dato y dos referencias, sin importar cuántos hijos tenga; el Heap no tiene nodos, porque vive en un arreglo contiguo que se duplica cuando se llena. Un evento está referenciado desde dos lugares, el árbol de su episodio y el AVL global de eventos, y un episodio desde tres, pero el objeto es uno solo en memoria: lo que se duplica es la referencia, no la información. El costo espacial adicional respecto del primer hito es, por lo tanto, el de los índices por fecha, que crecen en $O(E + V)$.

7. Decisiones tomadas en el Desafío 2

El Desafío 2 pidió reconocer la estructura jerárquica del problema y representarla como tal. La primera decisión fue identificar qué partes del escenario son realmente jerárquicas y cuáles no. El episodio lo es: cada intervención se desprende de otra y la profundidad no está acotada. El catálogo lo es por definición. En cambio, el registro de pacientes y la espera no son jerárquicos en el dominio, y allí los árboles se usan por su costo y no por lo que representan: el AVL porque acota la altura y el heap porque garantiza el mínimo en la raíz. Esta distinción es la que ordenó el resto del trabajo.

Otra decisión fue definir qué debía ocurrir con las funcionalidades del primer hito. Registrar, buscar, eliminar, agregar a la espera y consultar al próximo se mantienen con el mismo nombre y el mismo comportamiento observable; darDeAlta e ingresarPaciente se reemplazaron por cerrarEpisodio y abrirEpisodio o atenderSiguiente, porque su significado cambió al aparecer el episodio. La cola de prioridad se reimplementó con la estructura del Desafío 1 tal como exige la letra, y se agregó cambiarPrioridad como operación de la sala.

Respecto a los insumos, costos y tiempos, se decidió que se registren en el evento donde se consumen y que el episodio no guarde ningún total. Un total almacenado habría que mantenerlo actualizado en cada nivel de la jerarquía y se prestaría a duplicaciones; en cambio, calcularlo con un recorrido en postorden del subárbol pedido permite informar el episodio completo o cualquier parte de él con la misma operación. Los tiempos se calculan sobre los eventos cerrados, ya que un evento abierto no tiene duración definida.

La letra exige al menos cinco operaciones o consultas que requieran las estructuras desarrolladas y que no consistan en mostrar un dato almacenado, y pide que al menos tres de ellas sean consultas que el primer hito no podía responder sin recorrer la totalidad de los registros. Se eligieron las cinco que siguen, y para cada una se indica qué hace, qué estructura usa y por qué su resultado es un cálculo y no una lectura.

La primera es episodiosEnRango(desde, hasta), que devuelve los episodios de toda la sala abiertos en un rango de fechas. Recorre el AVL de episodios y descarta las ramas que quedan fuera del rango, en $O(\log E + k)$. Hay que decidir, para cada nodo, si su subárbol puede contener resultados antes de bajar a mirarlo, así que es un cálculo y no una lectura. En el primer hito la misma pregunta exigía recorrer la Pila completa.

La segunda es la sobrecarga episodiosEnRango(idPaciente, desde, hasta), que devuelve los episodios de un paciente puntual en un rango de fechas. Encadena dos podas: ubica al paciente en el AVL de registro, en $O(\log n)$, y dentro de su propio AVL de episodios poda de nuevo por el rango, en $O(\log e + k)$. En el primer hito no existía forma de aislar las atenciones de un paciente sin revisar todo el historial general.

La tercera es eventosEnRango(desde, hasta), que responde qué pasó entre dos fechas a nivel de cada atención individual, en lugar de por episodio completo, sobre el AVL global de eventos. En el primer hito ni siquiera existía el concepto de evento clínico, así que esta consulta no tenía forma de resolverse sin inventar una estructura nueva y recorrerla entera.

La cuarta es codigosDelCatalogo(codigo), que devuelve los códigos diagnosticables que cuelgan de un nodo del catálogo sin importar su nivel. Ubica el nodo en el árbol general y recorre su subárbol para devolver solo las hojas. Resuelve exactamente lo que pidió la dirección médica: consultar un capítulo completo sin enumerar a mano los códigos que contiene. Como en el primer hito no existía ningún catálogo, la única alternativa habría sido una lista plana filtrada entera en cada consulta.

La quinta es queImpideCerrarEpisodioDe(idPaciente), y su versión sobre un evento puntual, queImpideCerrarEvento. Recorre en postorden el subárbol de eventos y devuelve la lista de los que siguen abiertos y bloquean el cierre. Es un requisito explícito de la letra, que el sistema indique qué impide cerrar un episodio en un momento dado, y la respuesta depende de recorrer la jerarquía completa de derivaciones en el momento de la consulta. En el primer hito no existía la noción de episodio ni de eventos derivados unos de otros, así que esta pregunta no tenía sentido.

Para demostrar el funcionamiento de estas decisiones se escribieron pruebas orientadas a la solución, separadas de las que validan la biblioteca. EpisodioTest cubre el armado del árbol de derivaciones, el rechazo de eventos bajo un padre inexistente o cerrado, el cierre de abajo hacia arriba, el cálculo de costos y tiempos por subárbol y la reunión de códigos sin repetir.

CatalogoDiagnosticosTest verifica que un grupo no pueda colgarse de un código ni un código de un capítulo, y que codigosBajo devuelva lo que corresponde para cada nivel. TestSalaEmergencia recorre el ciclo completo de una atención, las consultas por rango con sus extremos incluidos, el cambio de prioridad y los casos de error de cada operación. A nivel de biblioteca, AVLTest incluye un caso que compara la altura de un ArbolBinarioBusqueda y de un AVLImpl tras la misma secuencia de inserciones ordenadas, para hacer visible la diferencia entre un árbol degenerado y uno balanceado, y HeapTest muestra que el orden de llegada no determina el de salida. En total el proyecto supera los trescientos casos de prueba.

8. Optimización realizada en el Desafío 3

El Desafío 3 pide identificar una operación de la solución del primer hito cuyo costo dependa del recorrido completo de una estructura, medirlo, proponer una alternativa basada en las estructuras del Desafío 1, implementarla y comparar ambas soluciones de forma experimental. A diferencia del primer hito, no se trata de buscar un cuello de botella hipotético: la solución entregada contenía operaciones cuyo costo crece con el tamaño total del sistema y que la nueva representación permite acotar.

La operación elegida es registrar y eliminar un paciente sobre el registro general de la sala. Se identificó revisando el código efectivamente entregado en el primer hito: buscarPaciente ya estaba optimizado con búsqueda binaria en esa entrega, pero registrarPaciente y eliminarPaciente seguían siendo $O(n)$. En registrarPaciente, el paciente nuevo se insertaba en la posición que le correspondía según el identificador mediante pacientesRegistrados.agregar(indice, paciente) sobre una ListaArray ordenada, y esa inserción desplaza todos los elementos posteriores, así que insertar cerca del principio cuesta $O(n)$. En eliminarPaciente, se llamaba a pacientesRegistrados.remove(paciente), que busca el elemento por objeto con indiceDe, un recorrido lineal completo del arreglo que compara con equals. Esto ocurre aunque el arreglo esté ordenado, porque la búsqueda es por igualdad de objeto y no por identificador, así que no aprovecha el orden ni la búsqueda binaria. Es un recorrido completo real, entregado en el primer hito.

El costo teórico de la versión del primer hito es el siguiente. registrarPaciente cuesta $O(n)$ en el peor caso, cuando se inserta cerca del principio y se corre todo el arreglo, y $O(1)$ en el mejor, cuando se inserta al final; registrar n pacientes en el peor orden posible cuesta $O(n^2)$ acumulado. eliminarPaciente cuesta $O(n)$ siempre, porque el recorrido lineal de indiceDe recorre hasta encontrar el elemento o hasta el final, y después el desplazamiento cubre lo que falta.

La alternativa propuesta fue reemplazar la ListaArray ordenada por un árbol AVL, AVLImpl<Paciente>. Esto ya estaba resuelto como parte del Desafío 2 y, la consigna permite reutilizar esta misma alternativa: registrar, buscar y eliminar pasan a ser de $O(\log n)$ en los tres casos, porque el AVL descende un solo camino acotado por su altura balanceada, sin recorrer nunca la estructura en su totalidad. La implementación también estaba hecha: SalaEmergencia.pacientesRegistrados es un AVLImpl<Paciente>, y tanto registrarPaciente como eliminarPaciente usan sus operaciones insertar y eliminar por criterio de identificador, sin escribir nada especial para el experimento.

El trabajo del desafío consistió entonces en la medición y la comparación. Se escribió un benchmark, BenchmarkRegistroPacientes, ubicado en src/main/java/ucu/edu/aed/benchmark, que arma una lista de identificadores con la propia ListaArray del proyecto, registra todos los pacientes y cronometra ese bloque con System.nanoTime(), y después los elimina a todos y cronometra ese otro bloque. El mismo archivo, sin cambios, se corrió contra el proyecto del primer hito (con ListaArray) y contra el proyecto actual (con AVL), para que la comparación sea contra la implementación efectivamente entregada y no contra una versión preparada para el experimento.

9. Resultados de los experimentos realizados

Los identificadores se generan con el formato P seguido de un número con ceros a la izquierda, de modo que el orden alfabético coincida con el numérico. Se probaron dos órdenes de llegada: al azar, mediante una permutación aleatoria de los números de 0 a $n-1$, que representa el caso promedio, y descendente, que es el peor caso para la ListaArray ordenada porque cada paciente nuevo se inserta al principio y desplaza a todos los anteriores. Se midieron volúmenes crecientes duplicando cada vez, 1000, 2000, 4000, 8000, 16000 y 32000 pacientes, con 7 repeticiones por combinación de volumen y caso, y se promediaron los tiempos. Cada repetición crea una SalaEmergencia nueva, registra los n pacientes y los elimina en el mismo orden en que los registró. Las mediciones del proyecto actual quedaron guardadas en resultados.csv, junto al benchmark. La siguiente tabla resume los promedios por volumen y caso.

Tabla 1. Proyecto actual (AVL): promedio de 7 repeticiones, en milisegundos.

Pacientes	Registrar, ids al azar	Registrar, ids descendentes	Eliminar, ids al azar	Eliminar, ids descendentes
1000	1,17	0,54	0,91	0,45
2000	1,02	0,57	0,88	0,45
4000	1,91	0,97	1,62	0,90
8000	4,12	1,74	3,63	1,78
16000	7,61	3,57	8,94	4,01
32000	19,10	7,42	19,91	8,57

Para el primer hito se conservaron los rangos observados entre repeticiones en los volúmenes de 1000, 8000 y 32000 pacientes, que son los que se usan en la comparación. La segunda tabla los contrasta con los rangos del proyecto actual en esos mismos volúmenes; para cada versión se toma el mínimo y el máximo entre todas las repeticiones de ambos casos, de modo que la comparación no dependa de una repetición favorable.

Tabla 2. Comparación entre el primer hito (ListaArray) y el proyecto actual (AVL), rango entre repeticiones, en milisegundos.

Pacientes	Registrar, hito 1	Registrar, actual	Eliminar, hito 1	Eliminar, actual
1000	1 o menos	0,2 a 4,5	1 o menos	0,2 a 2,8
8000	12 a 19	1,5 a 8,1	49 a 54	1,7 a 3,9
32000	361 a 532	6,6 a 23,0	1549 a 1959	8,1 a 20,9

A medida que crece el volumen, la diferencia se dispara. En 1000 pacientes ambas implementaciones están en el orden del milisegundo o menos, sin diferencia apreciable, y la primera repetición del proyecto actual es varias veces más lenta que las siguientes, lo que se atribuye al arranque de la máquina virtual de Java. Ya en 8000 pacientes registrar tarda entre 12 y 19 ms en el primer hito contra menos de 4 ms en el actual, si se deja de lado una repetición aislada, y eliminar entre 49 y 54 ms contra menos de 4 ms. En 32000 pacientes registrar tarda entre 361 y 532 ms en el primer hito contra 7 a 23 ms en el actual, y eliminar entre 1549 y 1959 ms contra 8 a 21 ms, una diferencia de dos órdenes de magnitud que, en la operación de eliminar, va de unas setenta veces a más de doscientas según qué repeticiones se comparen.

La forma de la curva confirma el análisis teórico. Del volumen 16000 al 32000, es decir al duplicar la cantidad de pacientes, el tiempo de eliminar en el primer hito casi se cuadruplicó, que es la firma de un costo $O(n^2)$ acumulado: el doble de pacientes, cada uno con un recorrido del doble de largo. En la versión actual el mismo salto de volumen llevó el promedio de eliminar de 8,9 a 19,9 ms con identificadores al azar y de 4,0 a 8,6 ms con identificadores descendentes, es decir, poco

más del doble, coherente con un costo $O(n \log n)$ acumulado. En términos absolutos el tiempo apenas se mueve, y la curva sigue siendo casi recta en la escala donde la del primer hito ya se dispara.

Hay un dato adicional que interesa para el análisis. En la versión actual, el caso denominado peor caso, con identificadores en orden descendente, salió sistemáticamente más rápido que el caso promedio, al revés de lo que ocurre en el primer hito. Esto se explica: la nueva estructura cambia la naturaleza del problema. Lo que era el peor caso para el arreglo ordenado, inserciones que van sistemáticamente al principio, no es un caso especialmente malo para el AVL, porque se reequilibra solo en cada inserción y el camino que recorre está acotado por la altura sin importar el orden de llegada. Que la secuencia ordenada resulte incluso algo más rápida se atribuye a que los identificadores consecutivos producen rotaciones regulares y accesos a memoria más predecibles que una permutación aleatoria, aunque esa diferencia no se midió por separado.

10. Declaración de uso de herramientas de IA

Durante el desarrollo del proyecto se utilizaron herramientas de IA generativa como apoyo en cinco aspectos puntuales, detallados a continuación manteniendo el criterio del primer hito: qué fue consultado, qué surgió de dicha consulta y qué se decidió por parte del equipo. En todos los casos el código y el texto incorporados fueron revisados, comprendidos y probados por el equipo, y las decisiones de modelado y de selección de estructuras fueron tomadas íntegramente por los integrantes.

El primer caso fue el recorrido por niveles. El equipo sabía que el recorrido debía visitar el árbol nivel a nivel y que la letra pedía apoyarlo en las estructuras lineales del primer hito, pero al equipo no se le había ocurrido cómo poder expresarlo sin recursión, así que se consultó a la herramienta cómo hacerlo. De esa consulta surgió la idea de usar una cola de nodos pendientes, que se implementó sobre la ListaEnlazada propia tanto en ArbolBinario como en ArbolGenerico. El método porNiveles fue luego revisado y probado por el equipo, incluyendo los casos de árbol vacío, un único nodo y un árbol degenerado.

El segundo caso corresponde a las pruebas unitarias. Se utilizó la herramienta para generar borradores de casos de prueba a partir de la interfaz de cada estructura y de la descripción del comportamiento esperado que el equipo le entregaba, en particular para cubrir los casos borde exigidos por la letra: estructura vacía, un único nodo, árbol degenerado y balanceado, y todos los casos del borrado en un árbol de búsqueda. Los casos fueron revisados uno por uno, y varios fueron corregidos o descartados cuando no reflejaban el comportamiento decidido por el equipo.

El tercer caso corresponde a los métodos toString, igual que en el primer hito: se utilizó apoyo para el formato de la salida, mientras que la información a mostrar fue definida por el equipo.

El cuarto caso corresponde al benchmark del Desafío 3, donde se consultó cómo medir el tiempo de un bloque de código en Java; de ahí surge el uso de System.nanoTime() y la conversión a milisegundos. La generación de los identificadores, los volúmenes, las repeticiones y los dos órdenes de llegada fueron definidos por el equipo.

El quinto caso corresponde a este informe. El borrador de la sección de principales decisiones de diseño se trabajó con asistencia de IA sobre decisiones que el equipo ya había tomado y discutido, para ordenar la redacción de argumentos que ya existían; el contenido fue revisado contra el código antes de incorporarlo. No se utilizaron herramientas de IA para el modelado del dominio, la elección de las estructuras de datos, la implementación de los TDA ni el análisis de complejidad presentado en este informe.

11. Conclusión

El desarrollo de este segundo hito permitió aplicar las estructuras jerárquicas a un escenario en el que la jerarquía forma parte del dominio, además de ser un recurso de representación. El episodio de atención, con sus derivaciones encadenadas de profundidad desconocida, y el catálogo de diagnósticos, con sus tres niveles fijos, se representaron con el mismo árbol general, pero con reglas distintas que hacen cumplir las clases que lo contienen. Esa separación entre la estructura y el dominio que la usa fue una de las decisiones que más ordenó el trabajo.

También, quedó en evidencia que al cambiar la representación de una colección existen consecuencias más allá de la operación que se buscaba mejorar. El pasaje de la ListaArray al AVL resolvió lo que pendiente en el primer hito, la inserción y la eliminación de pacientes y, la medición del Desafío 3 lo confirmó con diferencias de dos órdenes de magnitud en la operación de eliminar sobre 32000 pacientes. El pasaje de la ColaPrioridad al Heap abarató la inserción y habilitó el cambio de prioridad, pero costó el desempate por orden de llegada, y el análisis de complejidad mostró que algunas operaciones de la sala siguen siendo lineales por motivos ajenos a la estructura nueva, como el recorrido de Heap.remove al eliminar un paciente. Reconocer dónde está el costo dominante de cada operación fue, otra vez, lo que permitió justificar las decisiones y también sus límites.

También se aprendió a decir con precisión qué resuelve cada estructura. Las consultas por fecha bajaron de un recorrido completo a una búsqueda con poda porque el AVL tiene orden; las búsquedas en el catálogo siguen siendo lineales porque el árbol general no lo tiene, y lo que aporta es la organización por niveles, no la velocidad. Distinguir entre esas dos cosas evita atribuirle a un árbol propiedades que no posee.

Más allá de lo técnico, la experiencia de extender una solución propia sin romperla resultó distinta de la de construirla desde cero. Obligó a releer el código entregado en el primer hito con otros ojos, a decidir qué merecía conservarse y a aceptar que algunas partes que funcionaban bien, como el historial en una Pila, ya no representaban lo que el sistema necesitaba. Para el equipo, esa revisión crítica del propio trabajo fue tan formativa como la implementación de las estructuras nuevas.